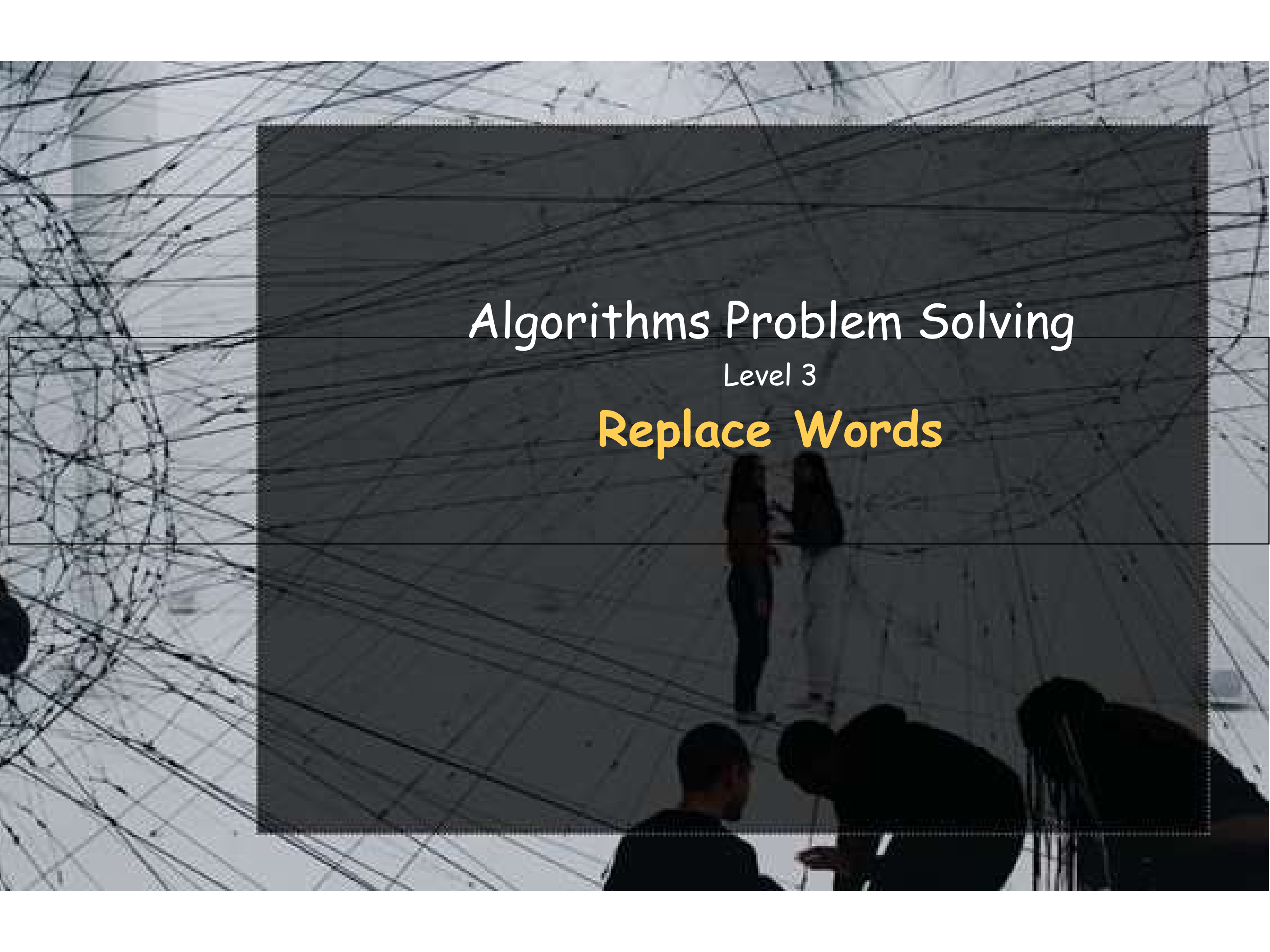


Algorithms Problem Solving

Level 3

Replace Words



Algorithms Problem Solving

Level 3

Replace Words

Definition

Substring-Based Word Replacement

 Program searches a sentence for occurrences of a target string and replaces them with another string.

 The implementation uses standard `find()` and `replace()` operations.

Explanation

Target Search

-  ``str.find(target)`` searches for the next occurrence of the target.

-  The returned position identifies where replacement should occur.

String Replacement

-  ``str.replace(pos, target.length(), replacement)`` removes the target substring and inserts the replacement.

-  The original string is modified directly.

Repeated Replacement

-  A loop continues searching until no additional occurrence is found.

-  This allows all matching occurrences to be replaced.

Procedures

- 1 Read the original sentence.
- 2 Read the target word.
- 3 Read the replacement word.
- 4 Search for the target.
- 5 Replace the matching substring.
- 6 Continue until no target remains.
- 7 Display the modified sentence.

💡 Example

String:

Welcome to Sweden, Sweden is nice.

Target:

Sweden

Replace:

Syria

➡ Result

Welcome to Syria, Syria is nice.

Note

- ◆ `find()` performs case-sensitive matching by default.
- ◆ This approach searches substrings rather than necessarily recognizing complete word boundaries.
- ◆ The same technique can be adapted for replacing phrases as well as individual words.

⊘ Warning

- ◆ Searching for `"Cat"` can also match `"Caterpillar"` because the operation is substring-based.
- ◆ Case differences prevent a match unless case-insensitive logic is added.
- ◆ Search position handling must be designed carefully when replacement text contains the target.



Comparison

▨ `ReplaceWord()` with `find()` / `replace()`

- ✓ Simple standard-library implementation.
- ✓ Works directly on the original string.
- ✓ Can replace arbitrary substrings.

▤ `ReplaceWordCustom()` with Tokenization

- ✓ Works on separate word tokens.
- ✓ Can enforce whole-word matching.
- ✓ Provides more control over case and punctuation handling.

✦✦ Summary

- ✓ Searches strings using ``find()`.`
- ✓ Replaces matching substrings using ``replace()`.`
- ✓ Can process multiple occurrences.
- ✓ Demonstrates direct modification of string content.

Thank you 😊